

1. Challenges in Power BI Dashboard Projects and Solutions

Challenge: Slow performance when refreshing a large dataset.

Solution:

- **Data Modeling:** Optimize the data model by reducing the number of columns and creating star schemas instead of snowflake schemas.
- **Aggregations:** Pre-aggregate data in the source or use Power BI aggregations for faster reporting.
- **DAX Optimization:** Replace complex DAX measures with simpler ones, or use calculated columns sparingly.
- **Query Reduction:** Disable "Auto Date/Time" and reduce the number of visuals or filters if not needed.

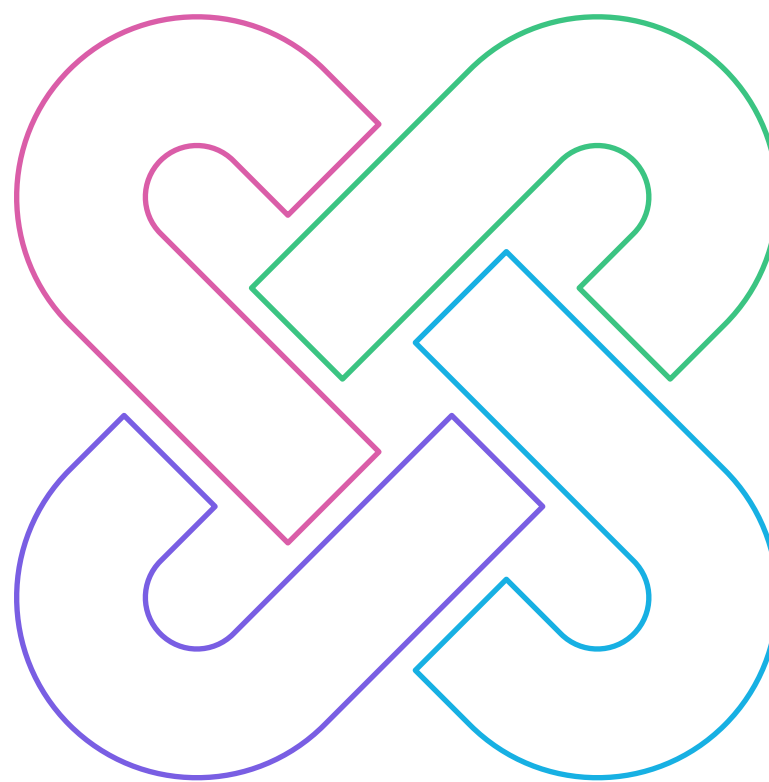
Optimizing Power BI Dashboard Performance

Query Reduction

Minimizing queries to enhance speed

DAX Optimization

Streamlining DAX measures for performance



Data Modeling

Simplifying data structures for efficiency

Aggregations

Pre-aggregating data for faster reporting

Practical Use Case:

Imagine a sales dashboard handling millions of records. Instead of showing granular transaction-level data, pre-aggregate sales data by month in the source, use summarized data in Power BI, and refresh only recent data rather than the entire dataset.

2. KPI vs. Dimension

- **KPI (Key Performance Indicator):** A KPI is a measurable value that indicates how well an objective is being achieved. Examples include revenue, profit margin, or customer churn rate.
- **Dimension:** A dimension provides context for analyzing KPIs. It's qualitative data such as time, region, product category, or customer segment.

Practical Example:

In a sales dashboard:

- **KPI:** Total Sales, Year-over-Year Growth
- **Dimension:** Region (North America, Europe), Time (Year, Quarter)

3. SQL Query to Find the Third Highest Salary

```
SELECT ESalary
FROM (SELECT ESalary, ROW_NUMBER() OVER (ORDER BY ESalary DESC) AS rank
      FROM Employee) AS ranked_salaries
WHERE rank = 3;
```

Explanation:

We use **ROW_NUMBER()** to rank salaries in descending order, then select the row where the rank is 3.

4. SQL Procedure to Select EIDs with Salary < 50,000

```
CREATE PROCEDURE GetEmployeesBelowSalary (@Salary INT)
AS
BEGIN
    SELECT EID
    FROM Employee
    WHERE ESalary < @Salary;
END;
```

Usage:

Execute with:

```
EXEC GetEmployeesBelowSalary 50000;
```

5. Retrieve EIDs with Odd Salaries and Join with empdetails

```
SELECT e.EID, d.EDOB
FROM Employee e
JOIN empdetails d ON e.EID = d.EID
WHERE e.ESalary % 2 = 1;
```

Explanation:

We check if **ESalary** is odd using **ESalary % 2 = 1** and join it with **empdetails** on **EID**.

6. Give Example of LAG in SQL

Using LAG for Week-over-Week Comparison

```
SELECT EID,
       ESalary,
       LAG(ESalary) OVER (PARTITION BY EID ORDER BY Week) AS PreviousWeekSalary,
       ESalary - LAG(ESalary) OVER (PARTITION BY EID ORDER BY Week) AS
WeekChange
FROM EmployeeSalaries;
```

Explanation:
LAG retrieves the salary from the previous week for comparison. This helps analyze week-over-week changes.

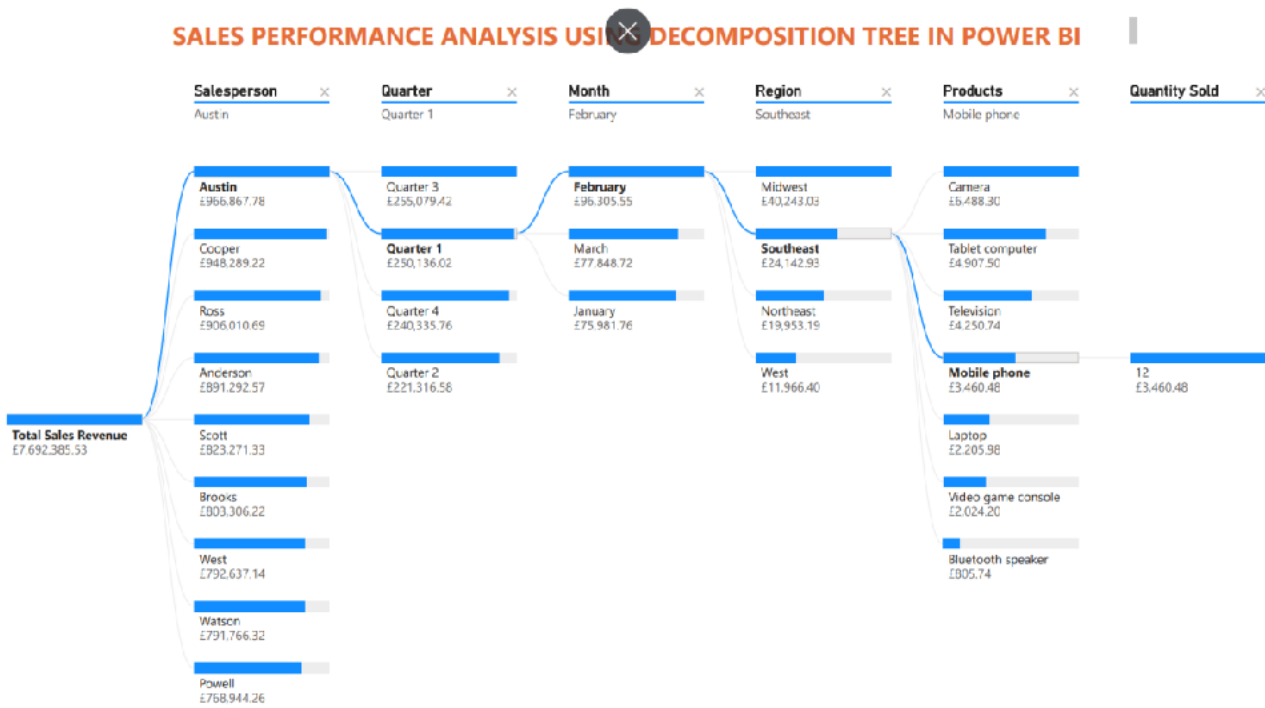
7. DAX Measure for Year-over-Year Growth

```
YoY Growth =
DIVIDE(
    [Total Sales] - CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date])),
    CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
)
```

Explanation:
This measure compares current sales with sales from the same period last year using **SAMEPERIODLASTYEAR**.

8. Unique Chart in Power BI: Decomposition Tree

Purpose:
The Decomposition Tree allows users to break down a KPI (e.g., Total Sales) into contributing factors (e.g., by Region, Product, or Salesperson). Users can drill down dynamically to explore hierarchies and see the impact of each category.



9. Implementing Time Intelligence in Power BI

Example: Analyze sales trends by month, quarter, or year.

- Steps:
- 1. **Create a Date Table** and mark it as the Date Table in Power BI.
 - 2. Use **DAX measures** for time intelligence:

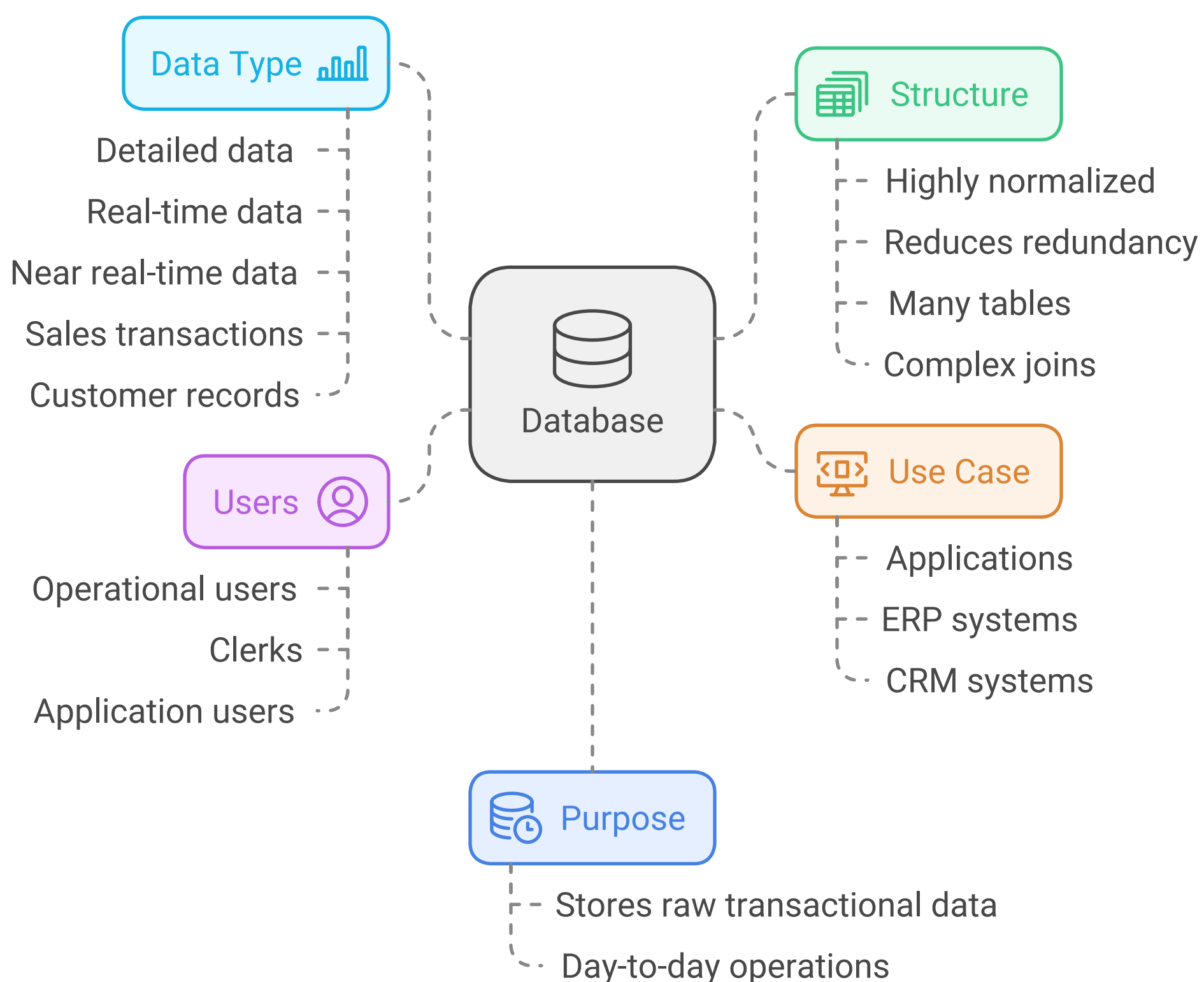

```
Total Sales YTD =  
CALCULATE([Total Sales], DATESYTD('Date'[Date]))
```

3. Create slicers for Year, Quarter, and Month to enable dynamic filtering.

10. What is the Difference Between a Database and a Data Mart?

Database:

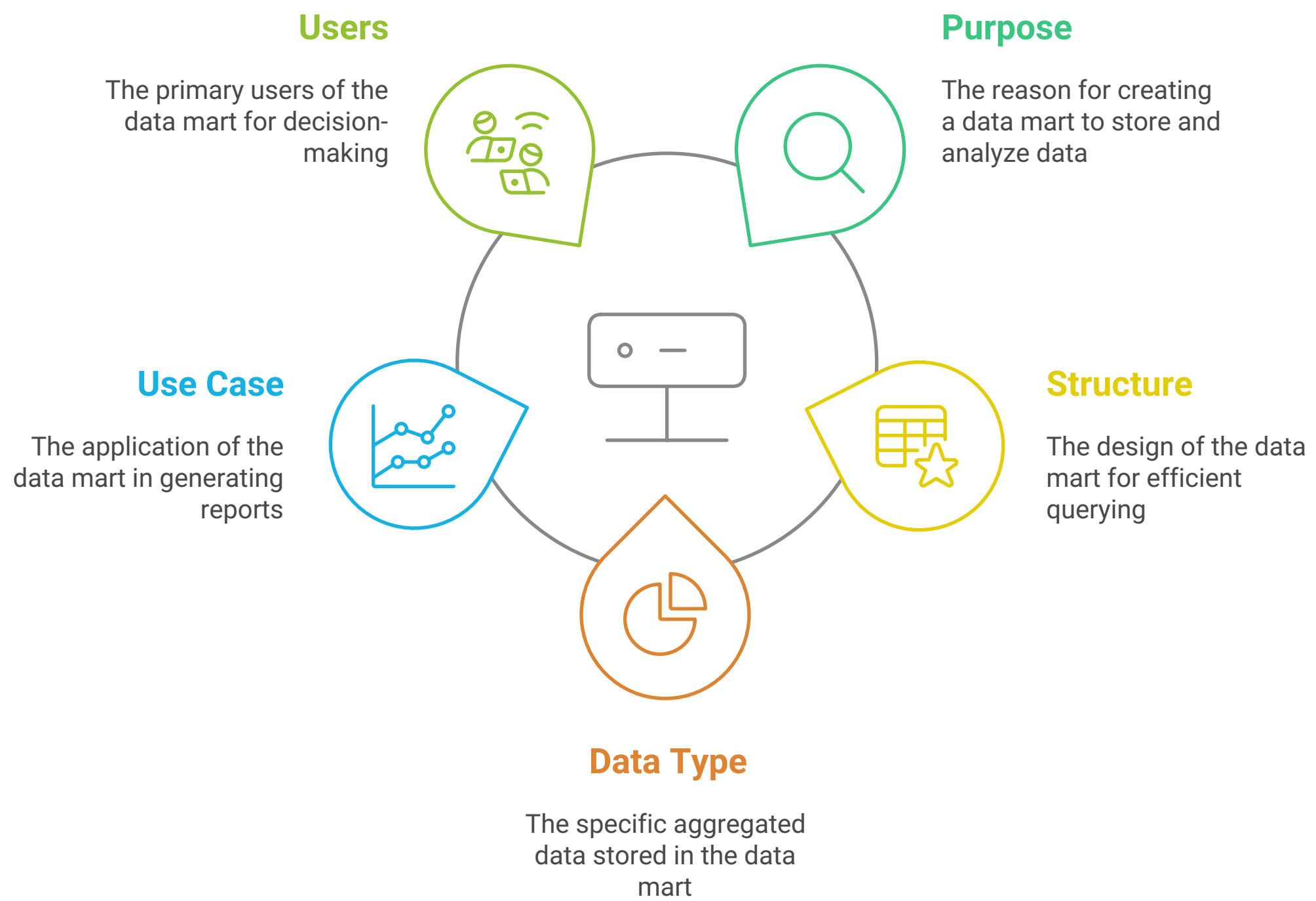
- **Purpose:** Stores raw transactional data used for day-to-day operations.
- **Structure:** Highly normalized to reduce redundancy (many tables, complex joins).
- **Data Type:** Detailed, real-time, or near real-time data such as sales transactions or customer records.
- **Use Case:** Used by applications, ERP systems, or CRM systems for operational purposes.
- **Users:** Operational users like clerks or application users.



Data Mart:

- **Purpose:** Stores summarized, subject-specific data for analytical and reporting purposes.
- **Structure:** Denormalized structure (fewer tables, star schema) optimized for fast querying.
- **Data Type:** Aggregated data focused on a specific business area like Sales, Marketing, or Finance.
- **Use Case:** Used to generate reports and dashboards for decision-making.
- **Users:** Business analysts and decision-makers.

Components of a Data Mart



Follow me on LinkedIn : <https://www.linkedin.com/in/rajatjain01/>

Subscribe to my Youtube Channel : <https://www.youtube.com/@rajatjain001>

Download Free Interview Questions PDF : https://topmate.io/rajat_jain01